

# Notes on Chapter 14: Strings

Norman Lim

2025-07-28

## Prerequisites

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.5.2      ✓ tibble     3.3.0
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.1.0
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(babynames)
```

## Creating a string

```
string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

## Escapes

```
double_quote <- "\"" # or "'"
single_quote <- "'" # or "\""
```

- Using a literal backslash to escape a backslash:

```
backslash <- "\\"
```

- To view strings properly, use `str_view()` :

```
x <- c(single_quote, double_quote, backslash)

str_view(x)
```

```
## [1] | '
## [2] | "
## [3] | \
```

## Raw strings

- Using multiple quotes and/or backslashes can be hard to read and confusing.
- Better method is to declare the string as “raw” string, just like in Python:

```
tricky <- r"(double_quote <- "\"" # or '''
single_quote <- '\'' # or """)"

str_view(tricky)
```

```
## [1] | double_quote <- "\"" # or '''
##      | single_quote <- '\'' # or ""
```

- Raw strings in R must be enclosed with `r"()"` or `r"[]"` or `r"{}"`

## Other special characters

- `\n` for new line.
- `\t` for tab.
- `\u` or `\U` for unicode escapes

```
x <- c("one\ntwo", "one\ttwo", "\u00b5", "\U0001f604")

str_view(x)
```

```
## [1] | one
##      | two
## [2] | one\ttwo
## [3] | μ
## [4] | 😊
```

- `str_view()` uses ``{}`` for tabs to make them easier to spot.

## Exercises

1. Create strings that contain the following values: > 1. He said “That’s amazing!” > 2. 3. \\

```
# No. 1
str_1 <- r("He said That's amazing!")
str_view(str_1)
```

```
## [1] | "He said That's amazing!"
```

```
# No. 2
str_2 <- r("\a\b\c\d")
str_view(str_2)
```

```
## [1] | \a\b\c\d
```

```
# No. 3
str_3 <- r("\\\\\\\\")
str_view(str_3)
```

```
## [1] | \\\\\\\
```

2. Create the string in your R session and print it. What happens to the special “0a0”? How does `str_view()` display it? Can you do a little googling to figure out what this special character is?

Ans.

```
str_4 <- "This\u00a0is\u00a0tricky"
str_view(str_4)
```

```
## [1] | This{\u00a0}is{\u00a0}tricky
```

The unicode character `u+00a0` ( `\u00a0` ) stands for no-break space. This is a space character that prevents an automatic line break at its position.

## Creating many strings from data

- `str_c` : takes any number of vectors as arguments and returns a character vector:

```
str_c("x", "y")
```

```
## [1] "xy"
```

```
str_c("x", "y", "z")
```

```
## [1] "xyz"
```

```
str_c("Hello, ", c("John", "Susan"))
```

```
## [1] "Hello, John" "Hello, Susan"
```

- `str_c()` is similar to `paste0()`, but is designed to be used with `mutate()`:

```
df <- tibble(name = c("Flora", "David", "Terra", NA))
df |>
  mutate(greeting = str_c("Hi ", name, "!"))
```

```
## # A tibble: 4 × 2
##   name greeting
##   <chr> <chr>
## 1 Flora Hi Flora!
## 2 David Hi David!
## 3 Terra Hi Terra!
## 4 <NA> <NA>
```

- `coalesce()` displays NA s in a different way:

```
df |>
  mutate(
    greeting1 = str_c("Hi ", coalesce(name, "you"), "!"),
    greeting2 = coalesce(str_c("Hi", name, "!"))
  )
```

```
## # A tibble: 4 × 3
##   name greeting1 greeting2
##   <chr> <chr>      <chr>
## 1 Flora Hi Flora! HiFlora!
## 2 David Hi David! HiDavid!
## 3 Terra Hi Terra! HiTerra!
## 4 <NA> Hi you! <NA>
```

## str\_glue()

- Using `str_c()` leads to a lot of " , making the code unreadable. `str_glue()` behaves in a similar way to f-strings in Python, but works in dplyr:

```
df |>
  mutate(greeting = str_glue("Hi {name}!"))
```

```
## # A tibble: 4 × 2
##   name greeting
##   <chr> <glue>
## 1 Flora Hi Flora!
## 2 David Hi David!
## 3 Terra Hi Terra!
## 4 <NA> Hi NA!
```

- One drawback of `str_glue()` is the way it handles NA s – `str_glue()` converts them to the literal string “NA”.
- To escape the curly brackets character “{” or “}” in `str_glue()` , you double it:

```
df |>
  mutate(greeting = str_glue "{{Hi {name}!}}"))
```

```
## # A tibble: 4 × 2
##   name greeting
##   <chr> <glue>
## 1 Flora {Hi Flora!}
## 2 David {Hi David!}
## 3 Terra {Hi Terra!}
## 4 <NA> {Hi NA!}
```

## str\_flatten()

- `str_flatten()` takes a character vector and combines each element of the vector into a single string:

```
str_flatten(c("x", "y", "z"))
```

```
## [1] "xyz"
```

```
str_flatten(c("x", "y", "z"), ", ")
```

```
## [1] "x, y, z"
```

```
str_flatten(c("x", "y", "z"), ", ", last = ", and ")
```

```
## [1] "x, y, and z"
```

- Using `str_flatten()` with `summarize()` :

```
df <- tribble(
  ~ name, ~ fruit,
  "Carmen", "banana",
  "Carmen", "apple",
  "Marvin", "nectarine",
  "Terrence", "cantaloupe",
  "Terrence", "papaya",
  "Terrence", "mandarin"
)

df |>
  group_by(name) |>
  summarize(fruits = str_flatten(fruit, ", "))
```

```
## # A tibble: 3 × 2
##   name      fruits
##   <chr>    <chr>
## 1 Carmen  banana, apple
## 2 Marvin   nectarine
## 3 Terrence cantaloupe, papaya, mandarin
```

## Exercises

1. Compare and contrast the results of `paste0` with `str_c()` for the following inputs:

```
str_c("hi ", NA)
str_c(letters[1:2], letters[1:3])
```

```
# str_c
str_c("hi ", NA)
```

```
## [1] NA
```

```
# str_c(letters[1:2], letters[1:3])

# paste0
paste0("hi ", NA)
```

```
## [1] "hi NA"
```

```
paste0(letters[1:2], letters[1:3])
```

```
## [1] "aa" "bb" "ac"
```

`str_c()` cannot throw an error running the second line due to “incompatible size”

# Extracting data from strings

## Separating into rows

- The most common tool is `separate_longer_delim()` :

```
df1 <- tibble(x = c("a,b,c", "d,e", "f"))
df1 |>
  separate_longer_delim(x, delim = ",")
```

```
## # A tibble: 6 × 1
##   x
##   <chr>
## 1 a
## 2 b
## 3 c
## 4 d
## 5 e
## 6 f
```

- Using `separate_longer_position()` :

```
df2 <- tibble(x = c("1211", "131", "21"))
df2 |>
  separate_longer_position(x, width = 1)
```

```
## # A tibble: 9 × 1
##   x
##   <chr>
## 1 1
## 2 2
## 3 1
## 4 1
## 5 1
## 6 3
## 7 1
## 8 2
## 9 1
```

## Separating into columns

- Using `separate_wider_delim()` :

```
df3 <- tibble(x = c("a10.1.2022", "b10.2.2011", "e15.1.2015"))
```

```
df3 |>
  separate_wider_delim(
    x, delim = ".",
    names = c("code", "edition", "year")
  )
```

```
## # A tibble: 3 × 3
##   code edition year
##   <chr> <chr>   <chr>
## 1 a10    1      2022
## 2 b10    2      2011
## 3 e15    1      2015
```

- Using the argument `NA` in `separate_wider_delim()` to omit a piece from the source vector :

```
df3 |>
  separate_wider_delim(
    x,
    delim = ".",
    names = c("code", NA, "year")
  )
```

```
## # A tibble: 3 × 2
##   code year
##   <chr> <chr>
## 1 a10   2022
## 2 b10   2011
## 3 e15   2015
```

- `separate_wider_position()` generally requires a known fixed set of columns:

```
df4 <- tibble(x = c("202215TX", "202122LA", "202325CA"))
df4 |>
  separate_wider_position(
    x,
    widths = c(year = 4, age = 2, state = 2)
  )
```

```
## # A tibble: 3 × 3
##   year age  state
##   <chr> <chr> <chr>
## 1 2022  15    TX
## 2 2021  22    LA
## 3 2023  25    CA
```



# Diagnosing widening problems

-If the width of a column is not known, the tools `too_few` and `too_many` can be useful:

```
df <- tibble(x = c("1-1-1", "1-1-2", "1-3", "1-3-2", "1"))
```

Running the code below will not work:

```
df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z")
  )
```

This code will throw the following error:

```
#> Error in `separate_wider_delim()`:
#> ! Expected 3 pieces in each element of `x`.
#> ! 2 values were too short.
#> i Use `too_few = "debug"` to diagnose the problem.
#> i Use `too_few = "align_start"/"align_end"` to silence this message
```

- We can use these error messages to debug our code:

```
debug <- df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_few = "debug"
  )
```

```
## Warning: Debug mode activated: adding variables `x_ok`, `x_pieces`, and
## `x_remainder`.
```

Looking at the contents of `debug` :

```
debug
```

```
## # A tibble: 5 × 6
##   x      y      z    x_ok x_pieces x_remainder
##   <chr> <chr> <chr> <lgl>    <int> <chr>
## 1 1-1-1 1      1    TRUE      3 ""
## 2 1-1-2 1      2    TRUE      3 ""
## 3 1-3    3    <NA> FALSE      2 ""
## 4 1-3-2 3      2    TRUE      3 ""
## 5 1      <NA> <NA> FALSE      1 ""
```

- The `x_ok` column tells us where the problem occurred:

```
debug |> filter(!x_ok)
```

```
## # A tibble: 2 × 6
##   x      y      z    x_ok x_pieces x_remainder
##   <chr> <chr> <chr> <lgl>    <int> <chr>
## 1 1-3    3    <NA> FALSE      2 ""
## 2 1      <NA> <NA> FALSE      1 ""
```

- The `x_pieces` column how many pieces were found. In this case we are expecting 3, but we only found 2 and 1 pieces. Now we know that the problem was due to having “too\_few” pieces. We can fill them with NA s to avoid R from throwing an error.
- We can then use the argument `too_few = "align_start"` or `too_few = "align_end"` to control where the NA s should go.

```
df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_few = "align_start"
  )
```

```
## # A tibble: 5 × 3
##   x      y      z
##   <chr> <chr> <chr>
## 1 1      1      1
## 2 1      1      2
## 3 1      3    <NA>
## 4 1      3      2
## 5 1    <NA> <NA>
```

What if we have too many pieces?

```
df <- tibble(x = c("1-1-1", "1-1-2", "1-3-5-6", "1-3-2", "1-3-5-7-9"))
```

- Using the argument `too_many` :

```
debug <- df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_many = "debug"
  )
```

```
## Warning: Debug mode activated: adding variables `x_ok`, `x_pieces`, and
## `x_remainder`.
```

```
debug |> filter(!x_ok)
```

```
## # A tibble: 2 × 6
##   x          y      z    x_ok x_pieces x_remainder
##   <chr>      <chr> <chr> <lgl>    <int> <chr>
## 1 1-3-5-6    3      5  FALSE      4 -6
## 2 1-3-5-7-9 3      5  FALSE      5 -7-9
```

- The `x_remainder` column also shows the excess pieces.
- We can opt to drop the excess pieces or merge them into the last column:

```
# dropping the excess pieces
df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_many = "drop"
  )
```

```
## # A tibble: 5 × 3
##   x      y      z
##   <chr> <chr> <chr>
## 1 1      1      1
## 2 1      1      2
## 3 1      3      5
## 4 1      3      2
## 5 1      3      5
```

```
# merging the excess into the last column
df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_many = "merge"
  )
```

```
## # A tibble: 5 × 3
##   x     y     z
##   <chr> <chr> <chr>
## 1 1     1     1
## 2 1     1     2
## 3 1     3    5-6
## 4 1     3     2
## 5 1     3    5-7-9
```

## Working with individual letters within a string

### Length

- Getting the string length using `str_length` :

```
str_length(c("a", "R for data science", NA))
```

```
## [1]  1 18 NA
```

- Using `str_length()` in dplyr:

```
babynames |>
  count(length = str_length(name), wt = n)
```

```
## # A tibble: 14 × 2
##   length      n
##   <int>   <int>
## 1     2   338150
## 2     3   8589596
## 3     4  48506739
## 4     5  87011607
## 5     6  90749404
## 6     7  72120767
## 7     8  25404066
## 8     9  11926551
## 9    10  1306159
## 10    11  2135827
## 11    12   16295
## 12    13   10845
## 13    14    3681
## 14    15     830
```

- Another example:

```
babynames |>
  filter(str_length(name) == 15) |>
  count(name, wt = n, sort = TRUE)
```

```
## # A tibble: 34 × 2
##   name      n
##   <chr>   <int>
## 1 Franciscojavier 123
## 2 Christopherjohn 118
## 3 Johnchristopher 118
## 4 Christopherjame 108
## 5 Christophermich  52
## 6 Ryanchristopher  45
## 7 Mariadelosangel  28
## 8 Jonathanmichael  25
## 9 Christianjoseph  22
## 10 Christopherjose  22
## # i 24 more rows
```

## Subsetting

- Using `str_sub()`:

```
x <- c("Apple", "Banana", "Pear")

str_sub(x, 1, 3)
```

```
## [1] "App" "Ban" "Pea"
```

- The length of the subset is equal to `end - start + 1`.
- Similar to Python, we can use `-1` to refer to the last character, `-2` to the second to last character, etc. (but this is not 0-indexed!)

```
str_sub(x, -3, -1)
```

```
## [1] "ple" "ana" "ear"
```

- But unlike Python, subsetting won't fail here if the string is too short – it will just return as much as possible.

```
str_sub("a", 1, 5)
```

```
## [1] "a"
```

- Using `str_sub()` in dplyr:

```
babynames |>
  mutate(
    first = str_sub(name, 1, 1),
    last = str_sub(name, -1, -1)
  )
```

```
## # A tibble: 1,924,665 × 7
##   year sex  name      n  prop first last
##   <dbl> <chr> <chr>   <int> <dbl> <chr> <chr>
## 1  1880 F    Mary    7065 0.0724 M    y
## 2  1880 F    Anna   2604 0.0267 A    a
## 3  1880 F    Emma   2003 0.0205 E    a
## 4  1880 F  Elizabeth 1939 0.0199 E    h
## 5  1880 F   Minnie   1746 0.0179 M    e
## 6  1880 F  Margaret  1578 0.0162 M    t
## 7  1880 F    Ida    1472 0.0151 I    a
## 8  1880 F   Alice   1414 0.0145 A    e
## 9  1880 F   Bertha  1320 0.0135 B    a
## 10 1880 F    Sarah  1288 0.0132 S    h
## # i 1,924,655 more rows
```

## Non-English text

- Using `charToRaw()` to convert a character to its hexadecimal form (aka the ASCII encoding):

```
charToRaw("Hadley")
```

```
## [1] 48 61 64 6c 65 79
```

- While ASCII coverage for symbols is limited, UTF-8 can encode just about every character, as well as other symbols, like smileys and other icons and shapes.
- UTF-8 can also “fail” when the input uses a different character encoding. Here is an example using `read_csv` from the **readr** package:

```
x1 <- "text\nE1 Ni\xf1o was particularly bad this year"
read_csv(x1)$text

x2 <- "text\n\x82\xb1\x82\xf1\x82\xc9\x82xbf\x82xcd"
read_csv(x2)$text
```

- We can specify the correct encoding via the `locale` argument.

```
read_csv(x1, locale = locale(encoding = "Latin1"))$text
read_csv(x2, locale = locale(encoding = "Shift-JIS"))$text
```

## Letter variations

- These two letters might look the same:

```
u <- c("\u00fc", "u\u0308")
str_view(u)
```

```
## [1] | ü
## [2] | ü
```

- But they differ in length:

```
str_length(u)
```

```
## [1] 1 2
```

- However, the string comparator function `str_equal()` from **stringr** still recognizes that both have the same appearance:

```
u[[1]] == u[[2]]
```

```
## [1] FALSE
```

```
str_equal(u[[1]], u[[2]])
```

```
## [1] TRUE
```

## Locale-dependent functions

- To check which locale is supported in **stringr**, use this command: `stringi::stri_locale_list()`.
- Example of difference in language rules depending on locale:

```
str_to_upper(c("i", "ı"))
```

```
## [1] "I" "I"
```

```
str_to_upper(c("i", "ı"), locale = "tr")
```

```
## [1] "İ" "I"
```

- Example of difference in sorting rules depending on locale:

```
str_sort(c("a", "c", "ch", "h", "z"))
```

```
## [1] "a" "c" "ch" "h" "z"
```

```
str_sort(c("a", "c", "ch", "h", "z"), locale = "cs")
```

```
## [1] "a" "c" "h" "ch" "z"
```

- The `arrange()` function in **dplyr** is also locale-dependent.